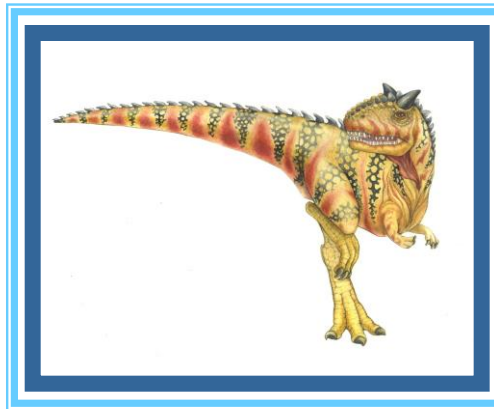


Chapter 7: Synchronization Examples

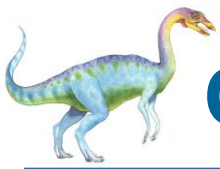




Outline

- Explain the bounded-buffer synchronization problem
- Explain the readers-writers synchronization problem
- Explain and dining-philosophers synchronization problems





Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem

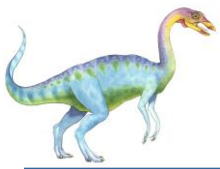




Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - **Readers** – only read the data set; they do **not** perform any updates
 - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities

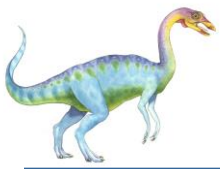




Readers-Writers Problem (Cont.)

- Shared Data
 - Data set
 - Semaphore **rw_mutex** initialized to 1
 - Semaphore **mutex** initialized to 1
 - Integer **readers_count** initialized to 0





Readers-Writers Problem (Cont.)

- The structure of a writer process

```
while (true) {  
    down(rw_mutex);  
  
    ...  
    /* writing is performed */  
  
    ...  
    up(rw_mutex);  
}
```





Readers-Writers Problem (Cont.)

- The structure of a reader process

```
while (true){
    down(mutex);
    readers_count++;
    if (readers_count == 1) /* first reader */
        down(rw_mutex);
    up(mutex);

    ...
    /* reading is performed */
    ...

    down(mutex);
    readers_count--;
    if (readers_count == 0) /* last reader */
        up(rw_mutex);
    up(mutex);
}
```





Readers-Writers Problem Variations

- The solution in previous slide can result in a situation where a writer process never writes. It is referred to as the “First reader-writer” problem.
- The “Second reader-writer” problem is a variation the first reader-writer problem that state:
 - Once a writer is ready to write, no “newly arrived reader” is allowed to read.
- Both the first and second may result in starvation, leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks





Dining-Philosophers Problem

- N philosophers' sit at a round table with a bowl of rice in the middle.



- They spend their lives alternating thinking and eating.
- They do not interact with their neighbors.
- Occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - Need both to eat, then release both when done
- In the case of 5 philosophers, the shared data
 - ▶ Bowl of rice (data set)
 - ▶ Semaphore chopstick [5] initialized to 1





Dining-Philosophers Problem Algorithm

- Semaphore Solution
- The structure of Philosopher i :

```
while (true){  
    down (chopstick[i] );  
    down (chopStick[ (i + 1) % 5] );  
  
    /* eat for awhile */  
  
    up (chopstick[i] );  
    up (chopstick[ (i + 1) % 5] );  
  
    /* think for awhile */  
  
}
```

- What is the problem with this algorithm?





Solutie corecta cu semafoare

```
#define THINKING      0
#define HUNGRY        1
#define EATING        2

int state[N];
semaphore_t  mutex = {1};
semaphore_t  ph[N]; // toate initializate cu 0
void philosopher(int i) {
    while(true) {
        think();
        take_forks(i);
        eat();
        put_forks(i);
    }
}
```





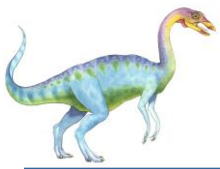
Solutie corecta cu semafoare

```
void take_forks(int i) {  
    down(mutex);  
    state[i] = HUNGRY;  
    test(i);  
    up(mutex);  
    down(ph[i]);  
}
```

```
void put_forks(int i) {  
    down(mutex);  
    state[i] = THINKING;  
    test((i - 1) % N);  
    test((i + 1) % N);  
    up(mutex);  
}
```

```
void test(int i) {  
    if(state[i] == HUNGRY &&  
        state[(i-1)%N] != EATING) &&  
        state[(i+1)%N] != EATING)  
    {  
        state[i] = EATING;  
        up(ph[i]);  
    }  
}
```





Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
{
    enum {THINKING; HUNGRY, EATING} state [5];
    condition self [5];

    void pickup (int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) self[i].wait;
    }

    void putdown (int i) {
        state[i] = THINKING;
        // test left and right neighbors
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}
```





Solution to Dining Philosophers (Cont.)

```
void test (int i) {
    if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
        state[i] = EATING ;
        self[i].signal () ;
    }
}

initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
}
```





Solution to Dining Philosophers (Cont.)

- Each philosopher “i” invokes the operations **pickup()** and **putdown()** in the following sequence:

```
DiningPhilosophers.pickup(i);
```

```
/** EAT **/
```

```
DiningPhilosophers.putdown(i);
```

- No deadlock, but starvation is possible

